

Centro Universitário UNA

UC – Modelos, Métodos e Técnicas da Engenharia de Software

Documento Técnico — Estratégia de Testes

Projeto: Nyox Imóveis

Versão do Documento: 1.0

Integrantes

- Carlos Eduardo da Silva
- Gustavo Henrique dos Santos
- Heitor Zeferino Siqueira
- Henrique Oliveira Ferreira
- João Vitor Martins Matos
- Pedro Henriques Ferreira
- Rodrigo Queiroz Vieira Freire

Estratégia de Testes

Este documento apresenta a estratégia de testes adotada em nosso monorepo, detalhando as ferramentas utilizadas e as métricas de cobertura obtidas no processo.

1. Abordagem Geral

Adotamos sempre que possível a prática de Test-Driven Development (TDD), mantendo como meta mínima **80% de cobertura de código**. Os testes são totalmente automatizados e integrados ao nosso pipeline de **Continuous Integration (CI)**.

2. Frontend (Aplicação React)

2.1. Tecnologias Utilizadas

- **Runner/Framework:** Vitest
- **Teste de Componentes:** React Testing Library
- **Asserções DOM:** `@testing-library/jest-dom`
- **Ambiente:** JSDOM
- **Cobertura:** `@vitest/coverage-v8`

2.2. Organização dos Testes

Os testes ficam no diretório `tests/`, espelhando a estrutura do diretório `src/`.

2.3. Como Executar

```
npm test
```

```
npm run test -- --coverage
```

2.4. Métricas de Cobertura (Frontend)

Resumo Geral:

- **Statements:** 89.28%
- **Branches:** 79.19%
- **Functions:** 89.84%
- **Lines:** 89.57%

Abaixo, uma visão reorganizada dos principais módulos:

Componentes

- **Alert:** 100%
- **AnuncioCard:** 100%
- **Carrossel:** 100%
- **Debounce (useDebounce):** 100%
- **Filtro:** 75.9%
- **Header:** 100%
- **MapView:** 92.85%
- **Sidebar:** 100%

Páginas

- **AnuncioPage:** 100%
- **CreateItem:** 98.33%
- **DeleteItem:** 97.77%
- **EditItem:** 88.88%
- **Home:** 100%
- **Login:** 75.51%
- **Main:** 84.84%
- **PainelAdm:** 92%

Arquivos `.css` não entram na cobertura.

3. Backend (API Node.js)

3.1. Tecnologias Utilizadas

- **BDD Runner:** Cucumber.js
- **Asserções:** Chai
- **HTTP Testing:** Supertest
- **Mocks de API:** Nock
- **Cobertura:** c8

3.2. Organização dos Testes

- `features/`: cenários em Gherkin
- `steps/`: implementação dos steps
- `serverInstance.mjs`: gerencia lifecycle do servidor

3.3. Como Executar

```
cd src/api  
npm test
```

```
cd src/api  
npx c8 npm test
```

3.4. Métricas de Cobertura (Backend)

Resumo Geral:

- **Statements:** 87.33%
- **Lines:** 87.33%

Detalhamento

- **api/cucumber.js:** 100%
 - **api/src/index.js:** 88.46%
 - **Config/Swagger:** 100%
 - **Connection/Supabase:** 100%
 - **Routes/Imoveis:** 88.57%
 - **Routes/User:** 89.14%
 - **Upload/imageUpload:** 13.79% (necessita testes)
-

Conclusão

O projeto mantém uma cobertura sólida tanto no frontend quanto no backend, atendendo a meta mínima estipulada e cobrindo maior parte das funções principais exigidas no sistema, ainda cabem melhorias e refatorações futuras no sistema.